

编程项目

截止时间：2025年5月19日星期一晚上11:59，在Gradescope上提交

1 简介

在这个项目中，你将构建一个匿名公告板，授权成员可以发布消息，但他们的发布不会透露是谁发布的。更具体地说，当你发布一个已签名的消息时，你可以选择一个匿名集，即一个可能已签名该消息的成员集合，包括你自己。任何人都可以验证消息确实是由该集合中的一个成员签名的，但无法确切知道是谁。你将使用一种酷炫的密码学原语——环签名来实现上述功能。

请注意这是一个编程项目，需要您编写可以在自己的笔记本电脑上运行的代码！如有任何关于设置初始代码和用Rust编程语言编程的技术问题，请向助教咨询。

2 环签名

环签名 [7, 4] 是一种密码学原语，它允许用户针对一组可能的签名者（包括实际签名者本人）对消息进行签名，使得签名在仅知道消息和签名者环的公钥集的情况下可以被验证——但它不会暴露实际签名者的身份。在这样的设置中，每个用户都生成一个公私钥对，并发布其公钥。我们将公钥视为该用户的身份²。

我们在此介绍环签名的语法。我们非正式地陈述了完整性、匿名性和不可伪造性，鼓励您阅读，但这些内容对项目并非至关重要。

定义1（环签名 [4]）环签名方案是一个PPT算法三元组 $(\text{RingGen}, \text{RingSign}, \text{RingVrfy})$ ，使得，

- $\text{RingGen}(\lambda)$ 在系统中为用户生成密钥对，其中 λ 是安全参数， pk 是用户的公钥， sk 是他们的私钥。
- $\text{RingSign}(pk, sk, R, m) \rightarrow \sigma$ 。使用公钥环 R 对消息 m 进行签名，其中 $R = \{pk_1, \dots, pk_n\}$ 是包含 pk 的公钥集合。
- $\text{RingVrfy}(R, m, \sigma) \rightarrow 0/1$ 。验证在消息 m 上声明的签名 σ ，相对于公钥环 R 。

¹ “环签名”中的“环”与数学上称为“环”的数学对象无关。它仅仅是一个集合。²我们假设存在某种公钥基础设施 (PKI) 来维护每个用户公钥的数据库。

参见图1。我们说该方案是完整的，如果对于由 n 调用 $\text{RingGen}(1^\lambda)$ 生成的每个集合 $\{(pk_i, sk_i)\}_{i \in [n]}$ ，对于每个 $j \in [n]$ ，对于每个包含 pk_j 的 $R \subseteq \{pk_1, \dots, pk_n\}$ ，以及对于每个消息 m ，都有 $\text{RingVrfy}(R, m, \text{RingSign}(pk_j, sk_j, R, m)) = 1$ 。非正式地说，我们说该方案是匿名的，如果签名隐藏了用于签署消息的用户秘密密钥。非正式地说，我们说该方案是不可伪造的，如果攻击者不能生成 (R, m, σ) 使得 $\text{RingVrfy}(R, m, \sigma) = 1$ 除非 (1) R 中的某个公钥是由攻击者生成的，或者 (2) 在此之前已经为 m 生成了有效的签名，相对于 R 。对完整定义感兴趣的学生可以参考 [4] 了解详细信息。

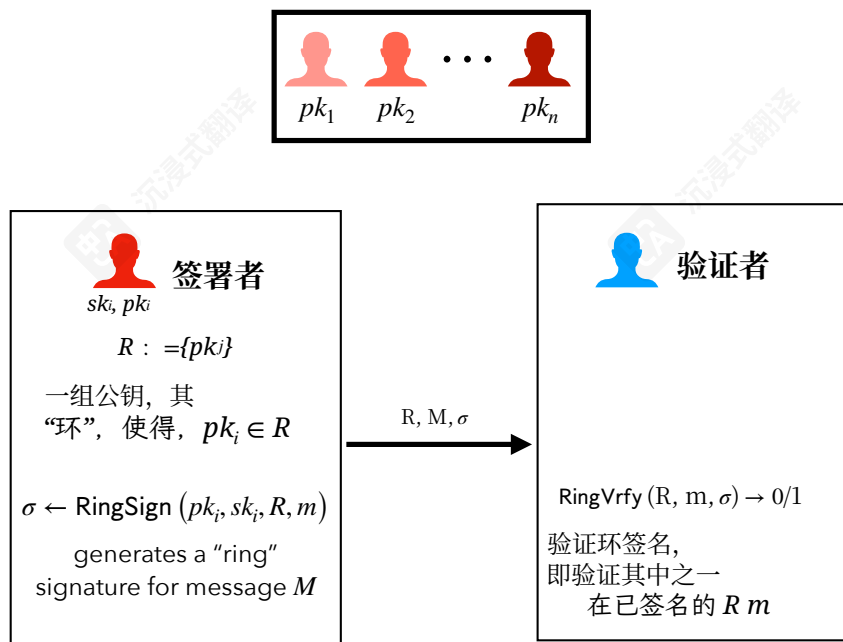


图1: 环签名语法。

2.1 使用 SNARK 构建环签名

在本项目中，我们将使用 SNARK 构建一种高效的环签名方案。我们的起点是一些安全的数字签名方案，例如 RSA 签名。我们简要回顾一下签名方案的语法 [5]。

定义 2 (数字签名) 签名方案是一个 PPT 算法三元组 $(Gen, Sign, Vrfy)$ ，使得，

- $Gen(1^\lambda) \rightarrow (pk, sk)$ (密钥生成)。 输出一个公钥验证 pk 和秘密签名密钥 sk 。
- $Sign(sk, m) \rightarrow \rho$ (签名)。生成消息 m 的签名 ρ ，使用密钥 sk 。
- $Vrfy(pk, m, \rho) \rightarrow 0/1$ (验证)。验证消息 m 上的签名 ρ 是否符合公钥 pk 。

我们假设签名方案是存在性不可伪造的，非正式地说，这意味着一个多项式时间 adversaries 无法伪造任何新消息（即，该 adversaries 之前未见过其签名的消息）的有效签名。

构建一个环签名。

令 Sig 是一种不可伪造的数字签名方案。

- 算法 $\text{RingGen}(1^\lambda)$ 仅运行 $\text{Sig.Gen}(1^\lambda)$ 并输出由此产生的 pk 和 sk 。
- 环签名算法 $\text{RingSign}(pk, sk, R, m) \rightarrow \sigma$ 使用 SNARK 证明者生成一个 ZK 证明 π ，证明它知道 (pk, ρ) ，使得 (1) $pk \in R$ 和 (2) $\text{Vrfy}(pk, m, \rho) = 1$ 。算法 $\text{RingSign}(pk, sk, R, m)$ 将此证明 π 作为环签名输出。
- 任何人都可以使用 SNARK 验证算法来验证环签名。公开声明是 (R, m) 这对。

更精确地说，你的任务是构建一个用于以下关系的 SNARK。

$$\mathcal{R} := \left\{ ((R, m), (pk, \rho)) \mid pk \in R \text{ and } \text{Sig.Vrfy}(pk, m, \rho) = 1 \right\}.$$

设 (P, V) 是 $\mathcal{R}$ 的 SNARK 证明者和验证者。图 2 和图 3 分别展示了环签名的构造和验证器的构造。

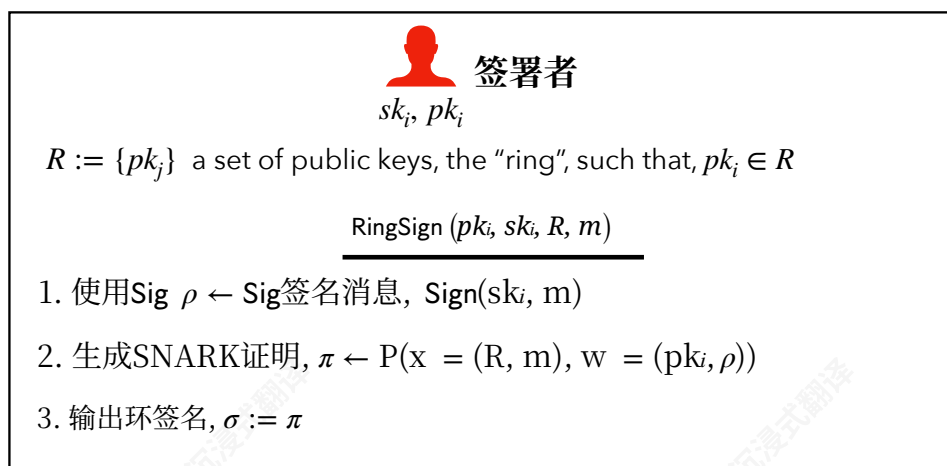


图 2：使用 SNARK 证明者的环签名签名者。

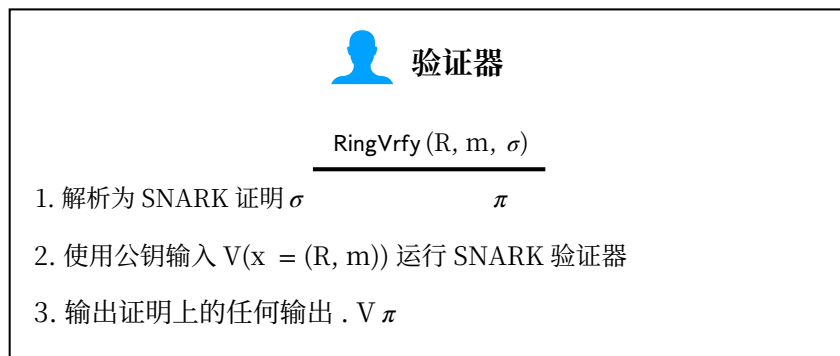


图 3: 使用 SNARK 验证器的环签名验证器。

在本项目中, 我们将使用名为 Plonky2 [1] 的开源库为 $\mathcal{R}$ 构建 SNARK。Plonky2 使用 PLONK 和 FRI (将在后续讲座中介绍) 来实现 SNARK。回想一下课堂内容, PLONK 允许我们为算术电路构建 SNARK。本质上, Plonky2 使开发者能够构建电路, 然后由库处理证明生成和验证。

你的任务是针对 $\mathcal{R}$ 在 Plonky2 中实现一个 SNARK!

首先, 我们需要一个具体的签名方案来实例化 Sig。

2.2 RSA 签名

RSA 签名方案是一种基于 RSA 假设难度的广泛使用的签名方案 [6]。下面, 我们将介绍如何实现这些算法。我们将固定公钥指数, $e = 2^{16} + 1 = 65537$ 。我们假设有一个哈希函数, 它将任意长度的字符串映射到 256 位, $H: \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ (例如 SHA256 或 Poseidon)。

定义3 (RSA签名方案) RSA签名方案由以下三种概率多项式时间 (PPT) 算法组成。

- $\text{Gen}(1^\lambda) \rightarrow (pk, sk)$
 1. 随机生成两个不同的较大 (例如 1024 位) 素数, p, q 。
 2. 计算 $N = pq$ 。计算欧拉函数³在 N 上, 即 $\phi(N) = (p-1)(q-1)$ 。
 3. 计算 $d = e^{-1} \pmod{\phi(N)}$ 。
 4. 输出 $(pk, sk) = (N, d)$ 。
- $\text{Sign}(sk = d, m) \rightarrow \rho$.
 1. 哈希消息, 哈希 $\leftarrow H(m)$ 。

³<https://zh.wikipedia.org/wiki/欧拉函数>

2. 使用PKCS1 v1.5方案填充哈希⁴,

$$\text{PaddedHash} := 0x00 \parallel 0x01 \parallel 0xFF \parallel \dots \parallel 0xFF \parallel 0x00 \parallel \text{hash}$$

使得结果长度等于 N 的大小。这里, $\parallel$ 指的是比特的连接, $0x00$ 、 $0x01$ 、 $0xFF$ 是十六进制中的字节。我们将填充后的哈希值视为 $\mathbb{Z}_N$ 的一个元素。

3. 输出签名, $\rho := (\text{PaddedHash})^d \pmod{N}$ 。

• $\text{Vrfy}(pk = N, m, \rho)$.

1. 按照上述定义计算 m 的填充哈希, PaddedHash_m 。

2. 输出 $\rho^e \stackrel{?}{=} \text{PaddedHash}_m \pmod{N}$

注意, SNARK 内部只需要实现 Vrfy 算法。我们已经为你提供了 Gen 和 Sign 的实现。

3 建立使用 Plonky2 库的 SNARKs

Plonky2 [1] 是一个由 Polygon Zero [2] 构建的开源加密库, 用于 SNARKs。它使用 Rust 语言构建, 这是一种流行的编程语言, 具有内置的内存安全、强静态类型和出色的静态分析器。我们建议通过《Rust by Example [3]》进行 Rust 的快速入门教程。我们还将涵盖完成第 (May 9) 节作业所需的所有基础知识, 请务必参加!

如前所述, 你的任务是构建 Plonky2 中的环签名验证电路。在本节中, 我们将简要介绍如何使用 Plonky2 构建电路。

3.1 电路构建器

在 Plonky2 中, 电路使用 CircuitBuilder 类构建。在启动代码中, 我们已经配置了 CircuitBuilder。使用构建器, 你可以通过添加输入线、构造门和指定输出条件来构建电路。电路中的每根线被称为“目标”, 电路通过对这些“目标”进行操作来构建。

3.2 大整数: BigUint

在之前的编程课程中, 你可能已经熟悉 uint 或无符号整数数据类型。在这个项目中, 我们将使用一种自定义数据类型 BigUint, 它是 uint 的一个强大但危险的亲戚。BigUint 支持对任意大的 (无符号) 整数进行算术运算! 出于安全考虑, 我们需要处理非常大的整数 (大约 2048 位), 这远远超出了任何原生数据类型能够存储的范围。Plonky2 使用无符号 32 位整数的向量来实现 BigUint。

3.3 目标

电路中的每一根线在 Plonky2 中均表示为“目标”。你的目标是使用适当的构造函数 (代表算术门) 来创建和连接目标。

⁴参考架构可以在以下位置找到: <https://datatracker.ietf.org/doc/html/rfc8017#section-9.2>。在这个项目中, 我们忽略算法 ID 的编码。

3.4 输入线

我们可以使用两个函数来指定电路的输入线,

1. `builder.add_virtual_public_biguint_target(num_limbs)`: 向电路添加一个用于 `BigUint` 的公共声明输入线, 其大小为 `num_limbs`。对于本项目, `num_limbs = 64`, 以便我们可以存储 2048 位无符号整数。输出一个 `BigUintTarget`, 即一个持有 `BigUint` 值的目标 (线)。
2. `builder.add_virtual_biguint_target(num_limbs)`: 向电路添加一个用于 `BigUint` 的证人输入线, 其大小为 `num_limbs`。输出一个 `BigUintTarget`。

3.5 常量

我们可以使用 `builder.constant(F::val)` 创建持有常量值的线值。对于本项目, 我们只需要 0 和 1 这两个值, 可以使用 `builder.zero()` 和 `builder.one()` 来创建。两者都输出一个 `Target` (不是大整数或布尔目标)。

3.6 相等门

我们可以使用 ``let output = builder.eq_biguint(biguint_target1, biguint_target2)`` 在电路中检查两个大整数目标 (`BigUintTarget`) 是否相等。它输出一个 `BoolTarget`, 这是一个布尔目标 (持有布尔值的线)。输出的 `BoolTarget` 在它们相等时持有值 1, 否则为 0。稍后, 我们需要将 `BoolTarget` 转换为“正常”的目标, 这可以通过使用 ``output.target`` 完成。

3.7 或门

给定两个布尔目标 (`BoolTarget`), 我们可以使用 ``builder.or(bool_target1, bool_target2)`` 计算它们的逻辑或值。它输出一个 `BoolTarget`, 与等式门类似。

3.8 与门

给定两个布尔目标 (`BoolTarget`), 我们可以使用 ``builder.and(bool_target1, bool_target2)`` 计算它们的逻辑与值。它输出一个 `BoolTarget`, 与或门类似。

3.9 加法门

我们可以使用 `builder.add_biguint(biguint_target1, biguint_target2)` 在电路中添加 `BigUint`。输出一个表示和的 `BigUintTarget`。

3.10 乘法门

我们可以在电路通过 `builder.mul_biguint(biguint_target1, biguint_target2)` 来乘以 `BigUint`。输出一个表示乘积的 `BigUintTarget`。

3.11 模运算门

我们可以在电路中通过 `builder.rem_biguint(biguint_target1, biguint_modulus_target)` 来对 `BigUint` 取模。输出一个表示 `biguint_target1 mod biguint_modulus_target` 值的 `BigUintTarget`。

3.12 输出条件

最后，Plonky2 使用 `builder.connect(target1, target2)` 定义输出条件。它接收两个类型为 `Target` 的目标，并检查它们是否相等。可以多次调用 `builder.connect` 来指定多个输出线的值。为了使电路评估为真，必须满足所有连接约束。可以将此视为检查每个输出线的正确性，并对所有约束进行逻辑与操作。

3.13 在证明生成中设置电路输入

构建电路后，证明者必须证明它知道电路的一些见证。Plonky2 模拟了证明生成过程，而您，开发者，只需设置每个输入线的值。这是在证明生成步骤中通过 `PartialWitness` 类完成的。它是“部分”的，因为它既包括公共声明，也包括见证输入。

```
let mut pw =部分证::new();
我们调用pw.set_biguint_target(&circuit_target, biguint_val)来将circuit_target的值设置
为BigUint值biguint_val。
```

一旦通过设置所有输入线（包括公理声明和证词）构建了部分证词，你就可以使用

```
let proof = circuit.circuit.prove(pw)?;
```

并使用

```
circuit.circuit.verify(proof);
```

请注意，你不需要自己调用这些函数。你只需要构建电路并设置输入值。我们在第4节中明确描述了启动代码和你的任务。

4 交付成果及提交要求

克隆以下 GitHub 仓库以开始使用。⁵

```
git clone https://github.com/entrophpy/CS355-project-ring-signature
```

你只需要编辑并提交文件 `./src/gadgets/rsa.rs`。你需要完成以下函数，

⁵如需任何帮助，请联系 Ed 或 OH 上的助教！

- **创建_环_电路**：此函数构建一个电路，用于捕获环签名关系 $\mathcal{R}$ 。
提示：你可能想直接编写检查关系的代码，但请记住你是在为关系构建电路，这意味着你需要在电路中编写检查关系的线缆和门 $\mathcal{R}$ 。
- **创建_环_证明**：使用构建的电路生成证明。您的任务是连接证明者的输入（公开声明和证人数值）到电路的适当输入线路上。
- **计算_填充_哈希**：给定消息的哈希作为输入，使用PKCS1 v1.5进行填充模式。请参考定义3中的RSA签名算法。这对于安全性是必需的！
- **幂_65537**：计算BigUIntTarget raised to the fixed power 65537 modulo some input modulus target。也就是说，您的目标是在电路中计算 $z^{65537} \pmod{N}$ 。

4.1 运行您的代码

准备好测试你的电路实现了吗？请按照以下步骤构建并运行你的 SNARK。

1. 生成 RSA 密钥对。

```
cargo run --release --example keygen
```

输出两个文件，key.json 和 key.pub.json，分别包含你的 RSA 秘密签名密钥和公钥验证密钥。

2. 编译证明者和验证者电路。

```
cargo run --release --example compile
```

将证明者和验证者电路编译成 circuit_prover.json 和 circuit_verifier.json。（请注意，证明者电路的规模相当大，但验证者电路要小得多。）

3. 生成证明。

```
cargo run --release --example prove <PATH_TO_PUBLIC_INPUT>  
<PATH_TO_PROVER_CIRCUIT> <PATH_TO_PUB_KEY> <PATH_TO_SEC_KEY>
```

使用生成的证明器电路生成保存在 proof.json 中的 SNARK 证明。公共输入文件包含环 R ，即用于签名的公钥集合，以及消息 m 。请参阅第 4.3 节，了解如何使用我们的在线演示生成此文件。

证明生成可能需要几分钟，具体取决于您的实现效率。完成后，它将输出一个文件，proof.json。

4. 验证证明。

```
cargo run --release --example verify <PATH_TO_VERIFIER_CIRCUIT>
<PATH_TO_PROOF> <PATH_TO_PUBLIC_INPUT>
```

如果证明有效，则输出成功。

4.2 测试

我们还为您提供了一些基本测试来验证您的实现。您可以使用 `cargo test --release` 运行所有测试。您可以通过在命令末尾附加测试名称来运行特定测试。

4.3 公告板演示

您已经编写了第一个 SNARK，太棒了！我们在

<https://web.stanford.edu/class/cs355/>

上举办了一个实时公告板，您可以使用您实现的环签名系统匿名发布消息！

您需要使用斯坦福大学的凭证来访问该页面。登录后，(1) 请在右上角通过上传您的公钥进行注册，(2) 上传您的验证电路。注册时请使用您的真实姓名。

现在您可以匿名发布信息了！选择您环中的用户，输入您的消息并下载证明输入。使用您的 SNARK 证明器，根据下载的公钥输入和编译的证明器电路生成证明，并将其上传到页面。接下来，您可以选择自己的电路或在课程中选择其他人的电路来验证它，并发布您的消息⁶！

4.4 在 Gradescope 上提交

您只需将完成的 `./src/gadgets/rsa.rs` 文件提交到 Gradescope，该文件将被评分。公告板上的活动不会被评分，但强烈建议您使用公告板来检查您的证明系统是否按预期工作。您不应编辑除 `./src/gadgets/rsa.rs` 之外的其他任何文件。

参考文献

- [1] 0xpolygonzero/plonky2. <https://github.com/0xPolygonZero/plonky2>.
- [2] Polygon zero. <https://polygon.technology/>.
- [3] Rust by Example. <https://doc.rust-lang.org/rust-by-example/>.
- [4] Adam Bender, Jonathan Katz, and Ruggero Morselli. Ring signatures: Stronger definitions, and constructions without random oracles. Cryptology ePrint Archive, Paper 2005/304, 2005.

⁶Ensure that your message adheres to the Fundamental Standard. See <https://communitystandards.stanford.edu/policies-guidance/fundamental-standard/fundamental-standard-may-1-2023>.

- [5] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2015. https://crypto.stanford.edu/~dabo/cryptobook/draft_0_2.pdf.
- [6] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, February 1978.
- [7] Ronald L. Rivest, Adi Shamir, and Yael Tauman. How to leak a secret. In Colin Boyd, editor, *Advances in Cryptology — ASIACRYPT 2001*, pages 552–565, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.